Controle Operacional Logístico

Modelagem em Teoria dos Grafos para Gestão Logística e Algoritmos de Menor Caminho e Tempo.

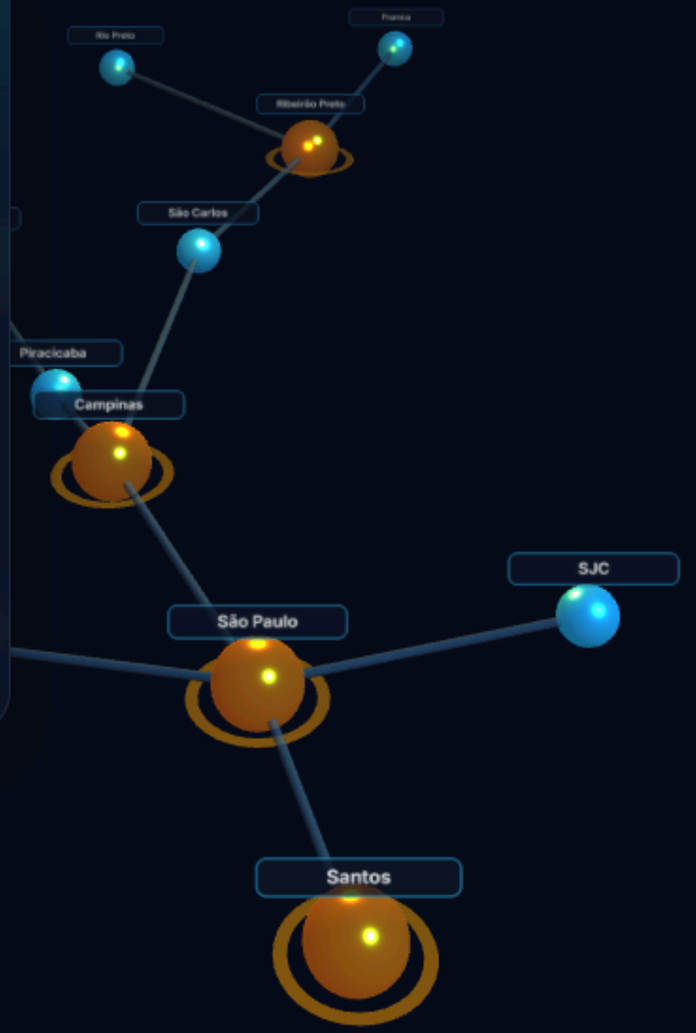
DISCIPLINA

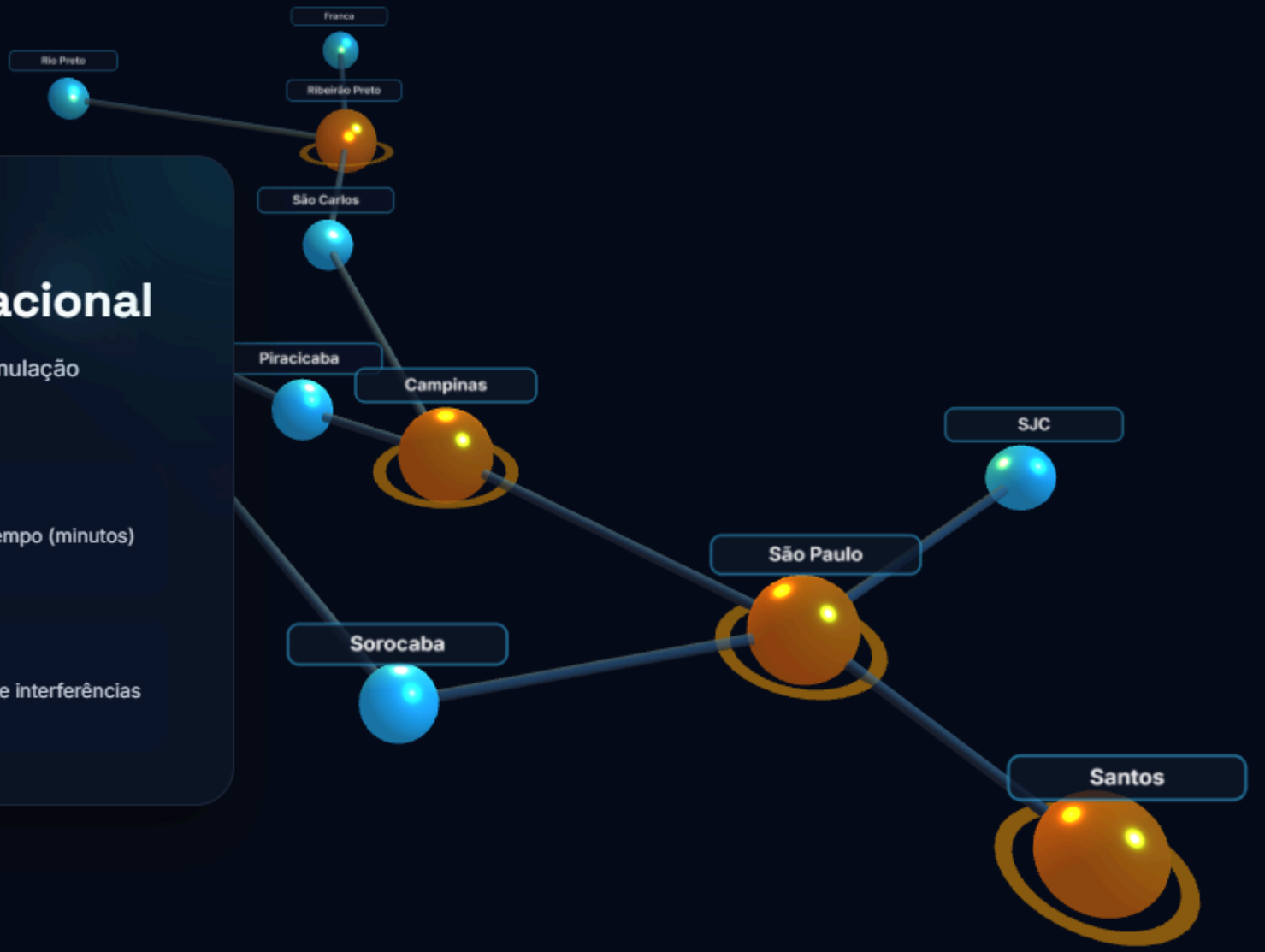
Engenharia da Computação · Teoria dos Grafos

Docente: Prof. Thiago Giroto Milani

EQUIPE DE DESENVOLVIMENTO

- Caroline Grizante (RA 114105)
- Emilly R. dos Santos (RA 114095)
- Guilherme Scaglia (RA 111598)
- Marcela Lovatto (RA 113626)





VISÃO GERAL

O Projeto Controle Operacional

Aplicação web focada em roteamento interativo e simulação operacional em tempo real.

Roteamento Interativo
Simula a mobilidade calculando dinamicamente o menor tempo (minutos) e a menor distância (km) na malha de transporte.

Central de Controle Operacional (CCO)
Integra restrições de veículos, habilitação, documentação e interferências climáticas em tempo real.



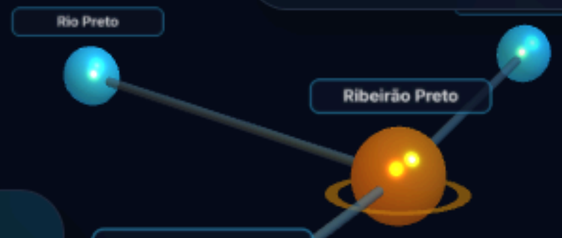
CONCEITO

Estrutura de Grafos: Nós e Arestas

Modelagem matemática da rede rodoviária do Estado de São Paulo.

Nós (Vértices)
Cidades, Centros de Distribuição (CDs), postos e terminais de carga com coordenadas lat/lng.

Arestas (Linhas)
Rodovias conectando os nós com distância (km), velocidade limite e pedágio base.







TOPOLOGIA

Tipo de Grafo: Dígrafo Ponderado

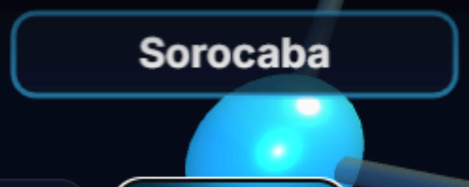
Fluxo direcionado e pesos dinâmicos ajustados em tempo de execução.

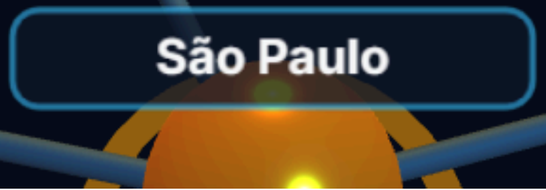
→ Grafo Direcionado (Dígrafo)

O sentido do fluxo é mandatório. A rota de A para B pode não ser igual no sentido inverso por conta de vias de mão única ou conversões restritas.

Grafo Ponderado

Cada aresta possui um peso que varia com o tipo do veículo, velocidade efetiva, condições climáticas e custos de pedágio por eixo.









PONDERAÇÃO

Ponderação Dinâmica: Tempo vs. Distância

Duas métricas distintas de otimização selecionáveis pelo usuário.

1. Menor Distância (km)
Soma geométrica do comprimento das arestas.

Soma Km

2. Menor Tempo (minutos)
Custo real no Dijkstra: `Tempo = (Km / Vel) * Fator`

Dijkstra min



AMBIENTE

Fatores do Ambiente e Operação

Variáveis conjunturais que afetam a velocidade média da malha.

Horário
Penalização por pico urbano.

Clima
Chuva (1.20x) ou Tempestade (1.60x).

Motorista
Fadiga e tempo de parada.



COMPLIANCE

Bloqueio Dinâmico e Restrições

Elevação do peso a Infinito (∞) ou remoção da aresta por não conformidade.

- **Capacidade & PBT:** Excesso de peso bruto total bloqueia vias urbanas.
- **Documentação & CNH:** CNH vencida ou categoria incompatível.
- **Certificação MOPP:** Obrigatória para transporte de cargas perigosas.

ARQUITETURA

Organização Modular do Projeto

Separação limpa entre Modelo de Dados, Controlador e Interface DOM.

data.js (Camada de Dados)
Estruturas de dados puras: CITIES, EDGES, VEHICLES, DRIVERS.

app.js (Controlador Operacional)
Dijkstra, validateCompliance(), drawNetwork() e manipuladores de eventos.

index.html & styles.css
Interface gráfica responsiva do painel e mapa Leaflet.



data.js ECMAScript 2024

```
// data.js - Estruturas de Dados do Grafo Completo

// 1. Vértices / Nós (Cidades e CDs)
const CITIES = [
  { id: 'SP', name: 'São Paulo', short: 'São Paulo', l
  { id: 'CAMP', name: 'Campinas', short: 'Campinas', l
  { id: 'RP', name: 'Ribeirão Preto', short: 'Ribeirão
  { id: 'SJC', name: 'São José dos Campos', short: 'SJ
  { id: 'SANT', name: 'Santos', short: 'Santos', lat:
];

// 2. Arestas / Conexões (Rodovias e Custos)
const EDGES = [
  { from: 'SP', to: 'CAMP', road: 'Anhanguera / Bandei
  { from: 'SP', to: 'SJC', road: 'Presidente Dutra', k
  { from: 'SP', to: 'SANT', road: 'Sistema Anchieta-Im
  { from: 'CAMP', to: 'RP', road: 'Washington Luís', k
];

// 3. Frota Operacional de Veículos
const VEHICLES = [
  { id: 'scania', name: 'Scania R450', capacity: 30000
  { id: 'sprinter', name: 'MB Sprinter', capacity: 150
```

Explicação técnica do bloco de código.



ALGORITMOS

Algoritmos Fundamentais Otimizados

Três algoritmos essenciais aplicados ao Grafo Operacional.

1. Algoritmo de Dijkstra (Caminho Mínimo)
Calcula a rota de menor tempo ou custo financeiro no grafo ponderado.

2. Busca em Largura (BFS)
Mapeia a quantidade mínima de saltos entre etapas de entrega.

3. Validação de Ciclos via DFS
Garante a topologia DAG evitando deadlocks operacionais.



CÓDIGO 1

Código: Inicialização e Desenho da Rede

Construção do Grafo visual usando a função drawNetwork() em app.js.

function drawNetwork()

Percorre o array EDGES desenhando conexões e o array CITIES adicionando marcadores com estilo dinâmico baseado em operador ternário para Centros de Distribuição.

</> app.js (drawNetwork) ECMAScript 2024

```
/* Desenha cidades, rodovias, pedágios, nomes e trânsi
function drawNetwork() {
  // ...
  EDGES.forEach(edge ⇒ {
    // ...
    const a = city(edge.from);
    const b = city(edge.to);
    const points = [[a.lat, a.lng], [b.lat, b.lng]];

    // Desenha a Aresta (Linha no Mapa)
    const roadLine = L.polyline(points, { color: '#e7a
      .bindTooltip(`${edge.road}<br>${edge.km} km · pe
    roadLayer.addLayer(roadLine);
    // ...
  });

  CITIES.forEach(c ⇒ {
    // Desenha o Nó (Ponto/Vértice no Mapa)
    const marker = L.circleMarker([c.lat, c.lng], {
      radius: c.cd ? 8 : 5, color: '#fff', weight: 2,
      fillColor: c.cd ? '#e7a51d' : '#1174b9', fillOpa
    }).bindPopup(`<strong>${c.name}</strong><br>Janela
    roadLayer.addLayer(marker);
```

Explicação técnica do bloco de código.



CÓDIGO 2

Código: Validação de Compliance

A função validateCompliance() atuando como primeira barreira de segurança.

`function validateCompliance(driver, vehicle, orders)`

Verifica se a CNH é válida, checa se a categoria atinge o peso do veículo com `categoryRank` e exige MOPP se houver carga perigosa.

</> app.js (validateCompliance) ECMAScript 2024

```
function validateCompliance(driver, vehicle, orders) {
  const errors = [];
  const dangerous = orders.some(order => order.danger)
  const categoryRank = { A: 1, B: 2, C: 3, D: 4, E: 5
  const weight = orders.reduce((sum, order) => sum + o
  const volume = orders.reduce((sum, order) => sum + o

  // 1. Validação de Documentos do Motorista e Veículo
  if (!driver.cnh) errors.push('CNH do motorista venci
  if (!vehicle.rntrc) errors.push('RNTRC do veículo in

  // 2. Validação de Categoria da CNH e Porte do Veícu
  if (vehicle.motorcycle) {
    if (driver.category !== 'A') errors.push('Categori
  } else if ((categoryRank[driver.category] || 0) < ca
    errors.push('Categoria de CNH incompatível com tra
  }

  // 3. Validação de Peso Limite por Categoria
  const categoryMaxWeight = { B: 3500, C: 10000, D: 10
  const maxWeight = vehicle.motorcycle ? Infinity : (c
  if (!vehicle.motorcycle && weight > maxWeight) {
    errors.push(`Peso incompatível com a categoria ${d
```

Explicação técnica do bloco de código.

São Carlos

Ribeirão Preto

CÓDIGO 3

Código: Ponderação Dinâmica no Dijkstra

Cálculo de peso em tempo de execução para cada nó vizinho.

Ponderação Interna do Dijkstra

Aplica o multiplicador `profile.factor` no tempo, ajusta pedágio pelo número de eixos e cancela a aresta se houver interdição.

</> app.js (dijkstra loop) ECMAScript 2024

```
// Dicionário de Perfis de Tráfego e Clima
const trafficProfiles = {
  normal: { label: 'Fluxo normal', factor: 1, level: '
  rain: { label: 'Chuva', factor: 1.20, level: 'warn'
  storm: { label: 'Tempestade', factor: 1.60, level: '
  accident: { label: 'Acidente', factor: 1.65, level:
  blocked: { label: 'Interdição', factor: 1, level: 'e
};

// Trecho interno do algoritmo de Dijkstra:
EDGES.filter(edge ⇒ edge.from ═══ current).forEach(ed
  // BLOQUEIO DINÂMICO DE VIA: Impede o uso de rodovia
  if (condition ═══ 'blocked' && blockedRoads.includes

  // 1. Ponderação de Tempo (Ajustada pelo Clima/Tráfe
  const profile = trafficProfiles[condition];
  const time = edge.km / edge.speed * profile.factor;

  // 2. Multiplicação do Pedágio pelos Eixos do Veícul
  const toll = edge.toll * (vehicle.axles / 2);

  // 3. Ponderação do Custo Total (Km + Valor/Hora do
  const cost = edge.km * (vehicle.costKm || 0) + time
```

Explicação técnica do bloco de código.



CÓDIGO 4

Código: Execução e Rastreamento da Rota

Reconstrução do caminho ótimo via ponteiro previous e dupla métrica.

Reconstrução & Acumulação

Usa um laço `while(cursor !== from)` desempilhando a rota com `.unshift()` e calcula distância e tempo de forma independente.

</> app.js (Reconstrução Dijkstra) ECMAScript 2024

```
function dijkstra(from, to, objective, condition, vehi
  // ... (Laço de exploração de nós)

  // Reconstrução da Rota a partir do destino
  const nodes = [to];
  const edges = [];
  let cursor = to;
  while (cursor !== from) {
    const edge = previous[cursor];
    if (!edge) return null; // Sem caminho viável
    edges.unshift(edge);
    nodes.unshift(edge.from);
    cursor = edge.from;
  }

  // Acumulação final dos dados da viagem
  let km = 0, time = 0, cost = 0, tolls = 0;
  edges.forEach(edge => {
    const profile = trafficProfiles[condition];
    const edgeTime = edge.km / edge.speed * profile.fa
    const edgeToll = edge.toll * (vehicle.axles / 2);

    km += edge.km;
```

Explicação técnica do bloco de código.

CONCLUSÃO

Comparativo Real vs. Limitações

O papel do modelo matemático e a atuação da Central de Controle Operacional.

Paralelo com a Vida Real

Simula a operação de uma CCO garantindo legalidade documental, capacidade física e segurança da carga.

Limitação Algorítmica

Algoritmos determinísticos não preveem 100% de imprevistos como falhas mecânicas repentinas ou erros humanos sem dados em tempo real.



ENCERRAMENTO

Muito Obrigado!

Agradecemos a atenção de todos. Estamos abertos a dúvidas e perguntas.

Perguntas & Respostas

Controle Operacional Logístico baseado em Teoria dos Grafos · Engenharia da Computação

